Chapter 5 — ML & Deep Learning Foundations

5.1 Tensors and Automatic Differentiation

Conceptual Deep Dive

A tensor is an n-dimensional array with a shape, dtype and device. Neural nets compose differentiable operations; **backpropagation** applies the chain rule backwards through the computation graph: for loss `L` and layer output $y = f(x, w)$, $\partial L/\partial w = \partial L/\partial y \cdot \partial y/\partial w$. Autograd records operations in the forward pass and replays them in reverse. Training loop: forward → loss → `backward()` → optimizer step (SGD, Adam: momentum plus per-parameter adaptive learning rates) → zero grads. Precision: fp32, **bf16** (wide range, preferred for LLM training), fp16 (needs loss scaling), fp8/int8/int4 for inference.

Cross-Connection Scenarios

LoRA (12) trains small matrices with the same machinery; quantisation (12, 13) trades precision for memory.

Production Anti-Patterns

- Forgetting `optimizer.zero_grad()` (gradients accumulate).
- Evaluating without `torch.no_grad()` / `model.eval()` (memory growth, dropout active).
- Mixing devices (CPU tensor with CUDA tensor).

Minimal Working Code

```
import torch

w = torch.tensor([2.0], requires_grad=True)
x, y = torch.tensor([3.0]), torch.tensor([10.0])
opt = torch.optim.SGD([w], lr=0.01)
for _ in range(200):
    opt.zero_grad()
    loss = ((w * x - y) ** 2).mean()
    loss.backward()
    opt.step()
print(w.item())  # approaches 10/3
```

5.2 Neural Network Essentials: Loss, Regularisation, Optimisation

Conceptual Deep Dive

Language models minimise **cross-entropy**: -log P(correct next token). **Perplexity** = exp(mean cross-entropy). Regularisation: dropout, weight decay, early stopping, data augmentation. Normalisation: LayerNorm / RMSNorm stabilise activations. Activations: ReLU, GELU, SwiGLU. Learning-rate schedules: warmup then cosine decay. Gradient clipping prevents explosions. Overfitting shows as train loss falling while validation loss rises.

Cross-Connection Scenarios

Fine-tuning hyperparameters (12); eval metrics (10) include perplexity for base models.

Production Anti-Patterns

- Tuning on the test set.
- No warmup with Adam at high LR.
- Judging a model by train loss alone.

Minimal Working Code

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 0.5, -1.0]])
target = torch.tensor([0])
loss = F.cross_entropy(logits, target)
print(loss.item(), "perplexity:", torch.exp(loss).item())

class RMSNorm(torch.nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.g, self.eps = torch.nn.Parameter(torch.ones(d)), eps
    def forward(self, x):
        return self.g * x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
```

5.3 Tokenization: BPE, WordPiece, SentencePiece, tiktoken

Conceptual Deep Dive

Models consume integer token ids. **BPE** (byte-pair encoding) starts from characters/bytes and repeatedly merges the most frequent adjacent pair until reaching a vocabulary size. **Byte-level BPE** (GPT-2, tiktoken) can encode any string with no unknown token. **WordPiece** (BERT) picks merges by likelihood gain and marks continuations with ##. **SentencePiece** treats text as a raw stream (spaces as _), supporting BPE or **Unigram LM**, and is language-agnostic. **tiktoken** is OpenAI's fast BPE implementation. Tokenizer quirks cause real bugs: numbers split unpredictably, whitespace matters, non-English text costs more tokens, glitch tokens, and each model family has its own vocabulary — token counts are not portable.

Cross-Connection Scenarios

Chunk sizes (9) should be measured in the target tokenizer; cost (2.4); fine-tuning needs chat-template-correct special tokens (12).

Production Anti-Patterns

- Counting tokens as len(text)/4 for billing-critical logic.
- Training with a different chat template than inference.
- Assuming one tokenizer for embeddings and generation.

Minimal Working Code

```
from collections import Counter

def train_bpe(corpus: list[str], num_merges: int):
    words = Counter(tuple(w) + ("</w>",) for line in corpus for w in line.split())
    merges = []
    for _ in range(num_merges):
        pairs = Counter()
        for word, freq in words.items():
            for a, b in zip(word, word[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        best = max(pairs, key=pairs.get)
        merges.append(best)
        new_words = Counter()
        for word, freq in words.items():
```

```
            out, i = [], 0
            while i < len(word):
                if i < len(word) - 1 and (word[i], word[i + 1]) == best:
                    out.append(word[i] + word[i + 1]); i += 2
                else:
                    out.append(word[i]); i += 1
            new_words[tuple(out)] += freq
        words = new_words
    return merges

print(train_bpe(["low lower lowest", "new newer newest"], 8))

# Production: import tiktoken; enc = tiktoken.get_encoding("cl100k_base"); len(enc.encode(text))
)
```

5.4 Attention and the Transformer Architecture

Conceptual Deep Dive

Self-attention lets each token mix information from others: Attention(Q,K,V) = softmax(QK^T/√d_k) V. The √d_k scaling keeps dot products from saturating the softmax. **Multi-head** attention runs h smaller attentions in parallel and concatenates, letting heads specialise. Decoder-only LLMs apply a **causal mask** so token i attends only to ≤ i. A transformer block = (norm → attention → residual) + (norm → MLP → residual). Positional information: sinusoidal, learned, **RoPE** (rotary, now dominant), ALiBi. Efficiency variants: **MQA/GQA** (share K/V heads to shrink KV cache), **FlashAttention** (IO-aware exact attention, avoids materialising the n×n matrix), sliding-window attention, Mixture-of-Experts (route each token to a few expert MLPs). Attention cost is O(n^2) in sequence length; that is why long context is expensive.

Architectures: encoder-only (BERT; embeddings/classification), decoder-only (GPT/Llama; generation), encoder-decoder (T5; translation/summarisation).

KV cache: during decoding, past keys/values are stored so each new token needs only its own Q against cached K,V. Cache size ≈ 2 × layers × kv_heads × head_dim × seq_len × bytes × batch; it, not weights, often limits concurrency.

Cross-Connection Scenarios

KV cache and GQA explain vLLM's PagedAttention (13); embeddings models (8) are encoders; context limits shape RAG (9).

Production Anti-Patterns

- Ignoring KV cache memory when sizing GPUs.
- Believing a larger context window means uniform recall (lost-in-the-middle).
- Forgetting the causal mask in custom code (information leak).

Minimal Working Code

```
import math
import torch
import torch.nn as nn

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).view(B, T, 3, self.h, self.dk).permute(2, 0, 3, 1, 4)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.dk)
```

```
        mask = torch.triu(torch.ones(T, T, dtype=torch.bool, device=x.device), 1)
        att = att.masked_fill(mask, float("-inf")).softmax(-1)
        return self.out((att @ v).transpose(1, 2).reshape(B, T, C))

x = torch.randn(2, 5, 32)
print(CausalSelfAttention(32, 4)(x).shape)

def kv_cache_bytes(layers, kv_heads, head_dim, seq, batch, bytes_per=2):
    return 2 * layers * kv_heads * head_dim * seq * batch * bytes_per
print(kv_cache_bytes(32, 8, 128, 8192, 1) / 1e9, "GB")
```

5.5 Pretraining, Scaling Laws and Post-Training

Conceptual Deep Dive

Pipeline: **pretraining** (next-token prediction on trillions of tokens) → **SFT** (instruction following) → **preference optimisation** (RLHF with PPO, or DPO/ORPO) → increasingly **RL with verifiable rewards** for reasoning models. **Scaling laws** (Kaplan; Chinchilla) say loss falls as a power law in parameters, data and compute; compute-optimal training uses roughly 20 tokens per parameter, though modern models overtrain smaller models for cheaper inference. Reasoning models spend extra tokens at inference ("test-time compute"). Hallucination arises because the objective rewards plausible continuations, not truth.

Cross-Connection Scenarios

Why RAG is needed (9); why evals matter (10); DPO details (12).

Production Anti-Patterns

- Expecting a base model to follow instructions.
- Assuming reasoning models are always better (cost/latency, overthinking).
- Treating the training cutoff as irrelevant.

Minimal Working Code

```
def chinchilla_tokens(params: float) -> float:
    return 20 * params

def training_flops(params: float, tokens: float) -> float:
    return 6 * params * tokens  # standard approximation

p = 7e9
print(f"{chinchilla_tokens(p):.2e} tokens, {training_flops(p, chinchilla_tokens(p)):.2e} FLOPs"
)
```

5.6 Hardware Acceleration: GPUs, TPUs and Memory Math

Conceptual Deep Dive

GPUs run thousands of parallel threads; matrix multiplies map to **tensor cores**. TPUs are matrix-multiply ASICs (systolic arrays) programmed through XLA/JAX. Constraints: **VRAM** (weights + KV cache + activations), **memory bandwidth** (decode is bandwidth-bound: tokens/s ≈ bandwidth / model bytes), **compute** (prefill is compute-bound), and **interconnect** (NVLink, InfiniBand) for multi-GPU. Parallelism: data, tensor, pipeline, expert, and ZeRO/FSDP sharding. Weight memory ≈ params × bytes-per-param (fp16: 2, int8: 1, int4: 0.5). Training with Adam needs ≈ 16 bytes/param without sharding (weights, grads, two moments in higher precision).

Cross-Connection Scenarios

Quantisation formats (12), serving engines (13), cloud GPU choice (13).

Production Anti-Patterns

- Sizing a GPU on weights only.
- Spreading a model across PCIe-connected GPUs and being surprised by latency.
- Leaving GPUs idle at low batch size.

Minimal Working Code

```
def weight_gb(params_b: float, bits: int) -> float:
    return params_b * 1e9 * bits / 8 / 1e9

def decode_tokens_per_s(bandwidth_gbs: float, model_gb: float) -> float:
    return bandwidth_gbs / model_gb

m = weight_gb(70, 4)
print(f"70B @4bit = {m:.0f} GB; ~{decode_tokens_per_s(2000, m):.0f} tok/s upper bound at 2 TB/s"
;)
```

5.7 Sampling and Decoding Strategies in Depth

Conceptual Deep Dive

Greedy decoding picks the argmax each step: deterministic but repetitive. **Beam search** keeps the B best partial sequences by cumulative log-probability (good for translation, poor for open-ended chat). Stochastic methods: temperature, **top-k**, **top-p (nucleus)**, **min-p** (keep tokens with probability at least p times the top token's, adapting to confidence), **typical sampling**, and penalties (repetition, frequency, presence). **Logit bias** nudges or bans tokens; **stop sequences** end generation; **logprobs** expose model confidence for calibration and classification (read the probability of the "yes" token). **Speculative decoding** accelerates without changing the distribution. Edge cases: temperature 0 is not bit-reproducible across hardware/batch sizes; very high top-k with high temperature yields incoherence; repetition penalties can break code and JSON.

Cross-Connection Scenarios

Self-consistency needs T>0 (6.2); classification via logprobs (10.3); constrained decoding masks logits (7.3); speculative decoding (13.1).

Production Anti-Patterns

- Tuning temperature and top-p simultaneously without isolating effects.
- Applying repetition penalties to structured output.
- Treating raw token probabilities as calibrated confidence.

Minimal Working Code

```
import math

def softmax(z, T=1.0):
    m = max(z)
    e = [math.exp((x - m) / T) for x in z]
    s = sum(e)
    return [x / s for x in e]

def min_p_filter(probs, p_base=0.1):
    thr = p_base * max(probs)
    kept = [p if p >= thr else 0.0 for p in probs]
    s = sum(kept)
    return [p / s for p in kept]

def top_k_filter(probs, k):
    cutoff = sorted(probs, reverse=True)[k - 1]
    kept = [p if p >= cutoff else 0.0 for p in probs]
```

```
    s = sum(kept)
    return [p / s for p in kept]

p = softmax([3.0, 2.5, 0.5, -1.0])
print(min_p_filter(p), top_k_filter(p, 2))
```

```
import math

def beam_search(step_fn, start, beam=3, steps=4):
    """step_fn(seq) -> list[(token, logprob)]."""
    beams = [([start], 0.0)]
    for _ in range(steps):
        cand = []
        for seq, score in beams:
            for tok, lp in step_fn(seq):
                cand.append((seq + [tok], score + lp))
        beams = sorted(cand, key=lambda x: -x[1])[:beam]
    return beams

toy = lambda seq: [("a", math.log(.6)), ("b", math.log(.3)), ("c", math.log
(.1))]
print(beam_search(toy, "<s>")[0])
```

5.8 Embedding Models, Encoders and Training Objectives

Conceptual Deep Dive

Encoder models (BERT-family) produce contextual token vectors; pooling (CLS, mean) yields a sentence vector. **Contrastive learning** (InfoNCE) with in-batch negatives and mined **hard negatives** trains retrieval embeddings; **cross-encoders** are trained with pairwise/listwise ranking losses; **distillation** compresses a cross-encoder's judgement into a bi-encoder. Decoder-LLM-based embedders (last-token pooling with instruction prefixes) now top benchmarks. Evaluation: MTEB/BEIR for orientation, your own recall@k for decisions. Edge cases: max sequence length truncation (silent), anisotropy (all vectors similar) fixed by normalisation or whitening, and multilingual mismatch between query and document language.

Cross-Connection Scenarios

Vector search (8), reranking (9.4), fine-tuning embeddings (8.6, 12).

Production Anti-Patterns

- Ignoring max length so long documents embed only their first 512 tokens.
- Choosing a model from MTEB rank without testing on your domain.
- No instruction/prefix when the model requires one.

Minimal Working Code

```
import torch
import torch.nn.functional as F

def info_nce(q: torch.Tensor, d: torch.Tensor, temperature: float = 0.05) -> torch.Tensor:
    """q[i] matches d[i]; all other d[j] in the batch are negatives."""
    q, d = F.normalize(q, dim=-1), F.normalize(d, dim=-1)
    logits = q @ d.T / temperature
    labels = torch.arange(len(q))
    return F.cross_entropy(logits, labels)

torch.manual_seed(0)
q, d = torch.randn(8, 32), torch.randn(8, 32)
print(info_nce(q, d).item())
```

```
import torch

def mean_pool(hidden: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    m = mask.unsqueeze(-1).float()
    return (hidden * m).sum(1) / m.sum(1).clamp(min=1e-9)

h = torch.randn(2, 5, 16)
mask = torch.tensor([[1, 1, 1, 0, 0], [1, 1, 1, 1, 1]])
print(mean_pool(h, mask).shape)
```

5.9 Positional Encodings and Long-Context Techniques

Conceptual Deep Dive

Attention is permutation-invariant, so position must be injected. **Sinusoidal** and **learned absolute** embeddings add a position vector to token embeddings. **RoPE** (rotary embeddings) rotates query/key pairs by an angle proportional to position, so the dot product depends on *relative* distance; it dominates modern LLMs. **ALiBi** adds a distance-proportional bias to attention scores and extrapolates to longer lengths. To extend context beyond training length: **position interpolation** (scale positions down), **NTK-aware scaling** and **YaRN** (adjust rotary base/frequencies), continued training on long data, and sparse/sliding-window attention. Long context costs O(n^2) compute and O(n) KV cache memory per sequence. Recall quality degrades in the middle ("lost in the middle") and with distractors, so benchmark with needle tests and realistic tasks, not just perplexity. Edge cases: quality drop past the trained length even when the API accepts longer input, KV cache dominating memory, and prefill latency growing quadratically.

Cross-Connection Scenarios

KV cache (5.4), RAG vs long context (9.1), context engineering (6.4), serving memory (13.1).

Production Anti-Patterns

- Feeding maximum-length prompts by default.
- Trusting extended-context models without a recall test on your data.
- Ignoring prefill latency in long-context SLOs.

Minimal Working Code

```
import torch

def rope_freqs(dim: int, base: float = 10000.0) -> torch.Tensor:
    return 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))

def apply_rope(x: torch.Tensor, positions: torch.Tensor, base: float = 10000.0) -> torch.Tensor:
    """x: (seq, dim) with even dim. Rotates consecutive pairs by position-dependent angles
."""
    freqs = rope_freqs(x.shape[-1], base)
    angles = positions[:, None].float() * freqs[None, :]
    cos, sin = angles.cos(), angles.sin()
    x1, x2 = x[..., 0::2], x[..., 1::2]
    out = torch.empty_like(x)
    out[..., 0::2] = x1 * cos - x2 * sin
    out[..., 1::2] = x1 * sin + x2 * cos
    return out

q = torch.randn(6, 8)
pos = torch.arange(6)
r = apply_rope(q, pos)
# relative-position property: dot(rot(q,i), rot(k,j)) depends only on i-j
print(r.shape, torch.allclose(apply_rope(q, pos).norm(dim=-1), q.norm(dim=-1), atol=1e-5))
```

```
def interpolate_positions(positions: torch.Tensor, trained_len: int, t
arget_len: int) -> torch.Tensor:
    """Position interpolation: squeeze longer positions into the trained range."
;"
    scale = min(1.0, trained_len / target_len)
    return positions.float() * scale

print(interpolate_positions(torch.tensor([0, 4096, 8191]), 4096, 8192))
```

5.10 Distributed Training, Mixture-of-Experts and Memory Optimisation

Conceptual Deep Dive

Parallelism strategies: **data parallel** (replicate model, split batch, all-reduce gradients), **tensor parallel** (split matrix multiplies across GPUs; needs fast interconnect), **pipeline parallel** (split layers into stages; micro-batching hides bubbles), **sequence/context parallel** (split long sequences), **expert parallel** (distribute MoE experts), and **ZeRO/FSDP** (shard optimiser state, gradients, then parameters across data-parallel ranks, cutting memory roughly by the number of GPUs). Memory savers: **mixed precision** (bf16), **gradient checkpointing** (recompute activations; ~30% slower, large memory savings), gradient accumulation, 8-bit optimisers, CPU/NVMe offload, and FlashAttention. **Mixture-of-Experts**: a router sends each token to top-k expert MLPs; total parameters grow while per-token compute stays modest; challenges are load balancing (auxiliary loss), communication cost, and memory for all experts. Edge cases: stragglers slowing synchronous training, non-deterministic reductions, checkpoint resharding between different GPU counts, and router collapse where a few experts get all tokens.

Cross-Connection Scenarios

Hardware math (5.6), LoRA/QLoRA (12.3), serving multi-GPU models (13.1, 13.9).

Production Anti-Patterns

- Using tensor parallelism across slow PCIe/Ethernet links.
- Training long runs without periodic checkpoints and resume tests.
- Ignoring expert load balance metrics.

Minimal Working Code

```
def training_memory_gb(params_b: float, zero_stage: int, n_gpus: int,
bytes_weights=2, adam_state=12):
    """Rough per-GPU model-state memory: bf16 weights+grads and fp32 master/Adam (12 B/par
am)."""
    p = params_b * 1e9
    weights, grads, opt = p * bytes_weights, p * bytes_weights, p * adam_state
    if zero_stage >= 1:
        opt /= n_gpus
    if zero_stage >= 2:
        grads /= n_gpus
    if zero_stage >= 3:
        weights /= n_gpus
    return (weights + grads + opt) / 1e9

for s in (0, 1, 2, 3):
    print(f"ZeRO-{s}: {training_memory_gb(7, s, 8):.1f} GB/GPU (7B, 8 GPUs, excludes activations)&qu
ot;)
```

```
import torch
import torch.nn.functional as F

def top_k_route(x: torch.Tensor, router_w: torch.Tensor, k: int = 2):
    """x: (tokens, d); router_w: (d, n_experts). Returns expert ids, weights, load-balance
```

```
loss."""
    logits = x @ router_w
    probs = F.softmax(logits, dim=-1)
    weights, ids = probs.topk(k, dim=-1)
    weights = weights / weights.sum(-1, keepdim=True)
    n_exp = probs.shape[-1]
    load = torch.zeros(n_exp).scatter_add_(0, ids.flatten(), torch.ones(ids.numel())) / ids.numel()
    importance = probs.mean(0)
    aux_loss = n_exp * (load * importance).sum()
    return ids, weights, aux_loss

ids, w, aux = top_k_route(torch.randn(16, 8), torch.randn(8, 4))
print(ids.shape, float(aux))
```

5.11 Multimodal Models and Diffusion Basics

Conceptual Deep Dive

Vision-language models typically pair a vision encoder (ViT/SigLIP/CLIP-style) with an LLM through a projector that maps image patch embeddings into the LLM's token space; images become hundreds to thousands of "visual tokens", driving cost. **CLIP-style contrastive training** aligns image and text embeddings in a shared space, enabling zero-shot classification and cross-modal retrieval. **Audio**: ASR (Whisper-style encoder-decoder) and TTS; native speech models skip text. **Diffusion models** generate images by learning to reverse a gradual noising process: sample noise, iteratively denoise guided by a text embedding (classifier-free guidance scales conditioning strength); latent diffusion works in a compressed autoencoder space for efficiency. **Video** models extend this over time. Practical concerns: image resolution vs token cost, OCR quality for dense documents, hallucinated visual details, and licensing/provenance of generated media. Edge cases: tiny text in screenshots, rotated or low-contrast images, and prompts embedded inside images (visual prompt injection).

Cross-Connection Scenarios

Multimodal RAG (9.6), multimodal prompting (6.8), voice assistant (17.8), injection via images (14.2).

Production Anti-Patterns

- Sending 4K images when a crop or downscale preserves needed detail.
- Trusting model-read numbers from charts without validation.
- Ignoring prompt injection hidden in images or PDFs.

Minimal Working Code

```
def image_token_estimate(width: int, height: int, patch: int = 14, mer
ge: int = 2) -> int:
    """Rough visual token count for a ViT-style encoder with token merging."""&qu
ot;
    return (width // patch // merge) * (height // patch // merge)

for size in [(448, 448), (1024, 768), (2048, 2048)]:
    print(size, image_token_estimate(*size))
```

```
import math

def cosine(a, b):
    return sum(x * y for x, y in zip(a, b)) / (math.sqrt(sum(x * x for x in a)) * math.sqrt(sum(y * y for
 y in b)))

def zero_shot_label(image_vec, label_vecs: dict[str, list[float]]):
    """CLIP-style: pick the label whose text embedding is closest to the image embedding.&
quot;""
    return max(label_vecs, key=lambda k: cosine(image_vec, label_vecs[k]))
```

```
print(zero_shot_label([1.0, 0.1], {"cat": [0.9, 0.2], "invoice": [0.0, 1.0]}))
```